

Chapter 15: Encapsulation in Java

Personal Notes

Encapsulation is one of the four pillars of Object-Oriented Programming. In simple words, it means hiding the internal data of a class and only allowing access to it through controlled, well-defined methods.

If you've ever wondered "why should I make fields private when I could just make everything public?" — this chapter answers that. Encapsulation is what makes Java code safe, predictable, and maintainable in real projects.

1. What is Encapsulation?

Encapsulation is the practice of:

- Making class variables PRIVATE so they cannot be accessed directly from outside
- Providing PUBLIC methods (getters and setters) to read or change those variables

This prevents direct access to an object's internal state and protects it from unintended or invalid changes. The class becomes the gatekeeper of its own data.

The Core Idea

Without Encapsulation		
+-----+		
	public balance = -9999	<-- anyone can write garbage
+-----+		
With Encapsulation		
+-----+		
	private balance	<-- locked inside
	public deposit(amount)	<-- controlled access
	public withdraw(amount)	(validates input)
	public getBalance()	
+-----+		

2. Why Encapsulation Matters

Encapsulation gives you three big wins in real code:

2.1 Data Hiding

By restricting direct access to variables, encapsulation ensures the internal state of an object stays safe and consistent. Other code can't reach in and corrupt the data.

2.2 Modularity

Changes inside a class don't ripple out to break other parts of the program. As long as the public methods stay the same, you can rewrite the internals freely. This is huge for maintaining large projects.

2.3 Security and Validation

Only specific, controlled operations can be performed on the data. You can validate every change inside setters — reject negative ages, invalid emails, oversized withdrawals — before they corrupt the object.

3. The 3 Steps to Implement Encapsulation

Implementing encapsulation in Java is always the same recipe:

1. Make all instance variables private
2. Provide public getter methods to read those variables
3. Provide public setter methods to update them — with validation if needed

Basic Template

```
public class Student {  
  
    // Step 1: private fields  
    private String name;  
    private int age;  
  
    // Step 2: public getters  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    // Step 3: public setters (with optional validation)  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setAge(int age) {  
        if (age > 0) {           // simple validation  
            this.age = age;  
        }  
    }  
}
```

```
}  
}  
}
```

Using the Encapsulated Class

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        s.setName("Aman");  
        s.setAge(19);  
  
        System.out.println(s.getName());    // Aman  
        System.out.println(s.getAge());    // 19  
  
        s.setAge(-5);                        // ignored – validation  
        rejected it  
        System.out.println(s.getAge());    // still 19  
  
        // s.name = "Hacker";    // ERROR – name is private  
    }  
}
```

Notice: Without the validation inside `setAge()`, someone could have set age to -5 and corrupted the object. Encapsulation gives you that gatekeeper power for every field.

4. Naming Convention for Getters & Setters

Java has a standard naming convention for getters and setters. Following it makes your code instantly readable to other developers and also lets frameworks like Spring detect them automatically.

Field	Getter	Setter
name (String)	getName()	setName(String name)
age (int)	getAge()	setAge(int age)
price (double)	getPrice()	setPrice(double price)
isActive (boolean)	isActive()	setActive(boolean active)

Boolean convention: For boolean fields, the getter is usually written as `isFieldName()` instead of `getFieldName()`. So it's `isActive()` not `getActive()` — reads more naturally in if-statements.

5. Practical Example: A Bank Account

This is the classic example that shows why encapsulation is non-negotiable in real code.

5.1 Without Encapsulation (BAD)

```
public class BankAccount {  
    public double balance;    // anyone can change this directly  
}  
  
// Usage:  
BankAccount acc = new BankAccount();  
acc.balance = 1000;  
acc.balance = -50000;        // valid Java – but a disaster!  
acc.balance = 999999999;     // free money? Sure!
```

There is no protection here. Anyone can assign any value — positive, negative, billions — and the account has no way to stop it.

5.2 With Encapsulation (GOOD)

```
public class BankAccount {  
    private double balance;    // hidden from outside  
  
    public BankAccount(double openingBalance) {  
        if (openingBalance >= 0) {  
            this.balance = openingBalance;  
        }  
    }  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        } else {  
            System.out.println("Invalid withdrawal");  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Using the Safe Version

```
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);

        acc.deposit(500);           // balance = 1500
        acc.withdraw(200);          // balance = 1300
        acc.withdraw(5000);         // Invalid – ignored

        System.out.println(acc.getBalance()); // 1300

        // acc.balance = -99999;    // ERROR – balance is private
    }
}
```

What changed? The balance field is private — no outside code can touch it directly. Every change goes through deposit() or withdraw(), which validate first. Bad inputs are rejected silently or logged. The class now protects itself.

6. The Simple Mental Model

Encapsulation works like a protective wrapper. Think of it as a sealed box or a locker:

- The data inside is protected from the outside world
- You can only interact with it through clearly defined openings (methods)
- The box decides what is allowed and what is not

Real-World Analogy: A TV Remote Control

+-----+	
⏻ CH▲ VOL▲	<-- public methods
⏻ CH▼ VOL▼	(buttons you press)
[1] [2] [3]	
+-----+	
hidden circuits	<-- private internals
+ microchips	(the user can't touch)
+-----+	

You interact with buttons (public methods). The internal circuits (private state and logic) stay hidden. You don't need to know HOW the remote works — only what it can do. That's encapsulation in everyday life.

7. Access Modifiers — Quick Recap

Encapsulation depends on Java's access modifiers. Here are all four:

Modifier	Where accessible	Typical use
private	Only inside the same class	Hide fields and helper methods
default	Inside the same package (no keyword)	Internal helpers within a package
protected	Same package + subclasses (even in other packages)	Available to children but not the world
public	Accessible from anywhere	Methods you want everyone to call

Rule of thumb: Default to private for fields. Default to public for methods that other code needs to call. Use protected only when you specifically want subclasses to access something. Avoid "default" unless you have a clear package-level reason.

8. Read-Only and Write-Only Fields

You don't always need BOTH a getter and a setter. The combinations you choose decide what the outside world is allowed to do.

8.1 Read-Only Field (getter only)

If you only provide a getter, the field can be read but never changed from outside. Useful for IDs, calculated values, or anything that should be set once and stay fixed.

```
public class User {
    private final int id;           // can never change after construction

    public User(int id) {
        this.id = id;
    }

    public int getId() {           // only a getter
        return id;
    }
}
```

8.2 Write-Only Field (setter only)

If you only provide a setter, the field can be set but never read directly. Useful for passwords or sensitive data — you write it, but the class never gives it back.

```
public class Account {
    private String password;
```

```

    public void setPassword(String password) {    // only a setter
        this.password = password;
    }

    public boolean checkPassword(String input) {
        return password.equals(input);           // no getter – never
    }
    expose it
    }
}

```

9. Benefits of Encapsulation

Benefit	How it helps
Data Hiding	Internal state stays private and safe from outside interference
Validation	Setters can reject invalid values before they corrupt the object
Maintainability	You can change the internal logic without breaking calling code
Flexibility	You can switch between fields and computed values transparently
Debugging	All changes go through methods — easy to log or trace
Security	Sensitive data can be hidden completely (write-only)
Testing	Easier to test classes that protect their own state

10. Common Mistakes

Mistake 1: Making fields public for "convenience"

```

public class Student {
    public int age;    // BAD – anyone can assign age = -100
}

```

Always start with private fields. Make them more accessible only if you have a real reason.

Mistake 2: Writing setters with no validation

```

public void setAge(int age) {
    this.age = age;    // BAD – accepts negative or unrealistic values
}

```

A setter without validation is barely better than a public field. Add the rules that matter for your domain.

Mistake 3: Returning mutable objects directly from getters

```

public class Cart {

```

```

    private List<String> items;

    public List<String> getItems() {
        return items;          // BAD – caller can now modify the internal
list
    }
}

```

If a field is a mutable object (list, array, date), the caller can change its contents even though the field is private. Return a defensive copy if you need full protection.

Mistake 4: Confusing encapsulation with abstraction

- Encapsulation = hiding the DATA (private fields + controlled access)
- Abstraction = hiding the IMPLEMENTATION (showing only what an object does, not how)

They are related but not the same. Encapsulation is the mechanism; abstraction is the design goal.

Mistake 5: Adding a getter/setter for EVERY field automatically

Just because a field is private doesn't mean it needs a getter and a setter. Add only what callers actually need. A field that's only used internally should stay completely hidden.

11. Best Practices

- Make all instance fields private by default
- Only expose getters/setters for fields that genuinely need outside access
- Add validation inside setters — never let invalid data slip in
- Use the standard getX() / setX() / isX() naming convention
- Prefer read-only fields (no setter) wherever possible — immutable objects are safer
- Return defensive copies of mutable internal objects from getters
- Combine encapsulation with constructors to ensure objects start in a valid state
- Never expose sensitive data (passwords, tokens) through getters

12. Quick Reference

Term	Meaning
private field	Hidden from outside the class
public method	Callable from anywhere
Getter (getX / isX)	Reads a private field
Setter (setX)	Writes to a private field — with validation
Read-only field	Has a getter, no setter

Term	Meaning
Write-only field	Has a setter, no getter
final keyword	Field can be set once and never reassigned
Defensive copy	Returning a copy instead of the original mutable object

13. Practice Problems

Try these to lock in the concept. Each one tests a slightly different aspect of encapsulation.

4. Create a Person class with private name and age. Add proper getters/setters with age validation (must be between 0 and 120).
5. Build an Employee class with id (read-only), name, and salary. Salary cannot be negative.
6. Design a Product class with private price field. Price must always be greater than 0 — reject 0 or negative values.
7. Create a User class with username (read-only) and a write-only password field. Add a checkPassword method.
8. Build a Rectangle class with private length and width. Add an area() method. Both dimensions must be positive.
9. Design a TempControl class with a private temperature field. The setter must reject anything outside the range -50 to 150.
10. Create a Cart class with a private list of items. Provide addItem(), removeItem(), and getItemCount() — but never expose the list directly.
11. Build a Date class with private day, month, year. Validate the values inside setters (no month = 13, etc.).

14. Final Takeaway

Encapsulation is not just a textbook concept — it is a practical, everyday approach to writing safer Java code.

- It improves maintainability by isolating changes inside a class
- It enhances code quality by enforcing validation through methods
- It promotes better design by separating internal logic from external access

Every serious Java codebase — from a small Android app to massive enterprise systems like Spring and Hibernate — relies on encapsulation as its foundation. Once you start writing classes this way, you'll never go back.

Key Takeaway: The rule is simple: keep fields private, expose only what's necessary, and validate everything that comes in. This single discipline removes an entire category of bugs and makes your code something other developers actually want to read.

15. What's Next?

Now that encapsulation feels natural, the next OOP topics worth learning are:

- Inheritance — building child classes from parent classes
- Polymorphism — method overloading and overriding
- Abstraction — abstract classes and interfaces
- The `this` and `super` keywords in depth
- Immutable classes — combining encapsulation with `final` fields
- Java Beans — a standard convention built on encapsulation